

Homework 2

End-to-end NLP System (RAG) Building

11-711 Advanced Natural Language Processing, Fall 2026

Released: Friday, September 25 **Due:** Friday, October 23, 11:59 PM ET

Individual assignment 100 points 10% of your grade

Large language models (LLMs) such as Llama 2 have been shown effective for question-answering (Touvron et al., 2023), however, they are often limited by their knowledge in certain domains. A common technique here is to augment an LLM’s knowledge with documents that are relevant to the question. In this assignment, you will *develop a retrieval augmented generation system (RAG)* (Lewis et al., 2021) that is capable of answering questions about Pittsburgh and CMU, including history, culture, trivia, and upcoming events.¹

Q: Who is Pittsburgh named after?

A: William Pitt

Q: What famous machine learning venue had its first conference in Pittsburgh in 1980?

A: ICML

Q: What musical artist is performing at PPG Arena on October 13?

A: Billie Eilish

So far in your machine learning classes, you may have experimented with standardized tasks and datasets that were easily accessible. However, in the real world, NLP practitioners often have to solve a problem from scratch (like this one!). This includes gathering and cleaning data, annotating your data, choosing a model, iterating on the model, and possibly going back to change your data. In this assignment, you’ll get to experience this full process.

There is no starter code. You must collect your own data and develop a model of your choice on the data. We provide a public leaderboard for you to evaluate your system before the final unseen test set. We will be releasing the inputs for the test set a few days before the assignment deadline, and you will run your already-constructed system over this data and submit the results. We also ask you to follow several experimental best practices, and describe the result in your report.

The key checkpoints for this assignment are:

- Understand the task specification
- Prepare your raw data
- Develop a retrieval augmented generation system with retrieval components
- Generate results
- Write a report
- Submit your work

On the use of AI. Per the syllabus, you may use AI to help you write *code*. You may **not** use it to write any part of your report.

¹See the [assignment policies](#) for this class, including submission information, late day policy and more.

Task: Retrieval Augmented Generation (RAG)

You'll be working on the task of factual question-answering (QA). We will focus specifically on questions about various facts concerning Pittsburgh and CMU. Since existing QA systems might not have the necessary knowledge in this domain, you will need to augment each question with relevant documents. Given an input question, your system will first retrieve documents and use those documents to generate an answer.

Preparing raw data

Compiling a knowledge resource

For your test set and the RAG systems, you will first need to compile a knowledge resource of relevant documents. We have provided a **baseline knowledge resource** that you may use as a starting point. You can build on top of this baseline by adding new sources and expanding coverage. You are free to use any publicly available resource, but we highly recommend including the websites below. Note that we can also ask you questions from relevant subpages (e.g. “about”, “schedule”, “history”, “upcoming events”, “vendors”, etc.) from these websites:

General Info and History of Pittsburgh/CMU

- Wikipedia pages ([Pittsburgh](#), [History of Pittsburgh](#)).
- [City of Pittsburgh webpage](#)
- [Encyclopedia Britannica page](#)
- [Visit Pittsburgh webpage](#): This website also contains subpages that would be useful for other topics (see below), like events, sports, music, food, etc.
- City of Pittsburgh [Tax Regulations](#): See the links under the “Regulations” column of the table
- City of Pittsburgh [2025 Operating Budget](#)
- [About CMU & CMU History](#)

Events in Pittsburgh and CMU (We will only ask about annual/recurring events and events taking place from **October 2026** onward.)

- [Pittsburgh events calendar](#): Navigate to month-specific pages for easier scraping
- [Downtown Pittsburgh events calendar](#)
- [Pittsburgh City Paper events](#)
- [CMU events calendar](#) and [campus events page](#)

Music and Culture (Note that many of these pages also contain upcoming events, also see Wikipedia pages for each.)

- Pittsburgh [Symphony](#), [Opera](#), and [Cultural Trust](#)
- Pittsburgh Museums ([Carnegie Museums](#), [Heinz History Center](#)), [The Frick](#), and [more](#)

Food-related events

- [Food Festivals](#)
- [Picklesburgh](#)
- [Pittsburgh Taco Fest](#)
- [Pittsburgh Restaurant Week](#)
- [Little Italy Days](#)
- [Banana Split Fest](#)

Sports (Note that many of these pages also contain upcoming events, also see Wikipedia pages for

each. Don't worry about scraping news/scores/recent stats from these sites.)

- General info ([Visit Pittsburgh](#))
- Pittsburgh [Pirates](#), [Steelers](#), and [Penguins](#)

Collecting raw data

Your knowledge resource might include a mix of HTML pages, PDFs, and plain text documents. You will need to clean this data and convert it into a file format that facilitates your model development. Here are some tools that you could use:

- To process HTML pages, you can use [beautifulsoup4](#).
- To parse PDF documents into plain text, you can use [pypdf](#) or [pdfplumber](#).

By the end of this step, you will have a collection of documents that will serve as the knowledge resource for your RAG system.

Training data

This is an **optional** aspect of this assignment, should you choose to train a model. The choice of training data is a bit more flexible, and depends on your implementation. If you are fine-tuning a model, you could possibly:

- Annotate it yourself manually, see the guidelines below.
- Do some sort of automatic annotation and/or data augmentation. You cannot use closed-source models (such as GPT 5, Claude etc.) for this process.
- Use existing datasets for transfer learning.

If you are using an LLM in a few-shot learning setting, you could possibly:

- Annotate examples for the task using the guidelines below.
- Use existing datasets to identify examples for in-context learning.

This training set you have constructed will constitute `data/train/questions.txt` and `data/train/reference_answers.json` in your submission. Read our model and data policy for this assignment.

Annotation guidelines (optional)

- **Domain Relevance:** Training examples should be similar in nature to the evaluation data (i.e., questions about Pittsburgh and CMU). Use the knowledge resources listed above when curating examples.
- **Diversity:** Your training data should cover a wide range of questions about Pittsburgh and CMU.
- **Size:** The size of your training data should be sufficient to support your training objective (e.g., fine-tuning or prompt selection). There is no minimum required size.
- If you want some guidelines about this, see the lecture on experimental design and evaluation.²
- **Quality:** Annotated examples should be accurate.

To help you get started, here are some example questions:

- Questions that could be answered by just prompting an LLM
 - Which three rivers meet in downtown Pittsburgh?
- Questions that can be better answered by augmenting an LLM with relevant documents

²Two lectures this semester cover this material: **Research Skills and Experimental Design** (October 8) and **Benchmarking and Evaluation Techniques** (October 22). Slides are posted on the [course schedule](#). Since the second falls the day before this assignment is due, you may also find the Fall 2025 notes on [experimental design](#) and [evaluation](#) useful earlier in the assignment.

- What is the name of the annual pickle festival held in Pittsburgh?
- Questions that are likely answered only through augmentation
 - When was the Pittsburgh Soul Food Festival established?
- Questions that are sensitive to temporal signals
 - Who is performing at X venue on Y date?

Developing your RAG system

Unlike assignment 1, there is no starter code for this assignment. You are free to use open source libraries and models. However, keep in mind the specific constraints outlined across different sections in this assignment, and make sure you provide due credit for any resources used in your report. See our model policy.

Implementation Requirements

For your RAG system, you will need the following three components:

1. **Document & query embedder** (can use existing models)
2. **Document retriever** (implement sparse, dense and hybrid retrieval)
3. **Document reader** (aka. question-answering system) (can use existing models)

For the core retrieval components listed below, you **MUST** implement the retrieval logic directly using libraries such as sentence-transformers, scikit-learn, numpy etc. You cannot use high-level RAG frameworks like LangChain, LlamaIndex, or similar tools that abstract away the retrieval implementation.

Document Chunking. Large documents need to be broken down into smaller, manageable pieces (chunks) for effective retrieval, since embedding models have token limits and retrieving overly large text segments can dilute relevance and overwhelm the generation model. Implement document chunking strategies that split documents into appropriately-sized segments while preserving semantic coherence. You may want to implement and try out different approaches (e.g., fixed-size with overlap, sentence-aware chunking, paragraph-based chunking etc.), since chunking strategy can have a significant impact on downstream retrieval and generation performance. Consider edge cases like very short documents, very long paragraphs, and documents with special formatting.

Hybrid Retrieval. Implement a system that combines dense (vector-based) and sparse (keyword-based) retrieval:

- **Dense Retrieval:** Use embedding models to create vector representations of documents and queries. We recommend starting with sentence-transformers models like `all-MiniLM-L6-v2` for efficiency, though you can experiment with other models (for guidance on selecting high-performing embedding models, refer to the [MTEB Leaderboard](#)) to improve performance. Use [FAISS](#) for efficient vector storage and similarity search. You'll need to implement the pipeline for embedding documents, building the FAISS index, and retrieving similar documents for queries.
- **Sparse Retrieval:** Integrate BM25 into your pipeline. You can use existing BM25 implementations, limited to [bm25s](#) or [rank-bm25](#), or build your own implementation if you wish to do so.
- **Combination Strategy:** Implement at least one established method for combining dense and sparse retrieval results. Common approaches include score normalization and weighted averaging, rank-based fusion methods (like Reciprocal Rank Fusion), using one method to filter or re-rank results from the other, or other ensemble techniques found in the literature.

Experiment with different combination strategies to see which works best for your dataset.

To get started with the overall RAG architecture, you can reference:

- 11-711 lecture notes
- [ACL 2023 tutorial on retrieval-augmented LMs](#)
- [llama-recipes](#) for an example RAG chatbot with Llama 2
- [Ollama](#) or [llama.cpp](#) to run LLMs locally on your machine

All the code for your data preprocessing, model development and evaluation will be a part of your GitHub repository (see submission for details).

Generating results

Finally, you will evaluate your systems on held-out test queries and submit your results for scoring. To support iterative development, we provide a public leaderboard to help you improve your system iteratively, as well as a separate unseen test set that will be used for final grading.

The final unseen test set (questions only) will be released the day before the assignment is due (**Thursday, October 22nd, 2026**).

Public evaluation data

We release `public_evaluation_data.json` alongside this handout: 157 questions about Pittsburgh and CMU, each with one or more reference answers, drawn from the same pool as the test set. Each entry looks like this:

```
{
  "id": "1",
  "question": "What open mic event occurs weekly on Mondays
              from 8:00-10:00 PM?",
  "answers": ["Monday Night Open Mic Comedy!"]
}
```

Because the reference answers are included, you can score this set locally with the metrics described below. Use it as your development set: it is how you are expected to run the comparisons that the analysis section asks for. Do not train or tune on it if you also plan to report it as a held-out number.

Note that this is **not** the leaderboard set. The leaderboard uses a different set of queries, and we will not release its reference answers.

Public leaderboard

The leaderboard is not live yet — we will post the leaderboard link along with queries (without answers) on Piazza as soon as it is released.

For this leaderboard:

- The queries are **different from** those in `public_evaluation_data.json`. Scoring well on the public evaluation data does not guarantee the same score here.
- We will **not** release the reference answers for the leaderboard queries, so you cannot score this set yourself.
- Only aggregate scores will be shown.
- You are limited to **at most 10 submissions** in total.

- You are only allowed to submit **one answer per question**.
- If you don't want your Andrew ID displayed, you can replace it with the nickname we assigned to you through email titled "Advanced NLP Assignment 2 Leaderboard Nicknames".

The leaderboard is intended solely as a development and diagnostic tool. Scores on the leaderboard will **not** be used directly for grading.

SUBMISSION FORMAT: PUBLIC LEADERBOARD

```
{
  "andrewid": <your Andrew ID or assigned nickname>,
  "1": "Answer 1",
  "2": "Answer 2",
  ...
}
```

Please make sure you follow this format. Points will be deducted for misformatted outputs.

Running your ablations

The leaderboard is **not** the place to run your ablations: you have at most 10 submissions in total, and the analysis section asks you to compare several systems (dense-only, sparse-only, hybrid, more than one fusion strategy, and retrieval vs closed-book). That is already more configurations than you have submissions.

Instead, run every comparison locally against `public_evaluation.data.json`, which ships with reference answers for exactly this purpose. If you annotate your own questions, holding out a portion of them gives you a second development set; never train, tune, or prompt-select on whatever you report as held-out. Report how many questions each number is computed over. Your development scores and the leaderboard scores measure different question sets, so keep them in separate columns of your results table and never average the two.

Use your leaderboard submissions sparingly, to sanity-check your best one or two systems against questions you did not write yourself.

Unseen test set

This test set will be curated by the course staff and will serve as the **final evaluation** of your system's ability to respond to a variety of questions about Pittsburgh and CMU. Because the goal of this assignment is not to perform hyperparameter optimization on this private test set, we ask you to not overfit to this test set. You are allowed to submit up to three output files (`system_outputs/system_output_{1,2,3}.json`). We will use the best performing file for grading.

SUBMISSION FORMAT: UNSEEN TEST SET

```
{  
  "1": "Answer 1",  
  "2": "Answer 2",  
  ...  
}
```

Please make sure you follow this format. Points will be deducted for misformatted outputs.

Release policy for late days

The [course late policy](#) applies: for each unexcused day your homework is late, 5% is subtracted from your grade for this homework. You may request excused late days by emailing f2026-11-711@andrew.cmu.edu at least 168 hours (7 days) before the deadline; medical requests with documentation can be made later.

This assignment adds one logistical requirement on top of that policy. Because the test set is released the day before the deadline, everyone who submits late must receive a **different** test set, so that students using late days do not gain extra time on the same questions. If you plan to submit late, you **MUST** tell the TAs via a Piazza post how many days you plan to take and the date you plan to submit. A day before that date, a different test set will be released to you. We have distinct test sets prepared for up to 5 late days.

Evaluation metrics

Your submissions will be evaluated on a combination of metrics, including standard metrics (answer recall, F1, and ROUGE-L) and LLM-as-Judge (score 1–5) with rubrics. See section 6.1 of the original SQuAD paper for details of standard metrics. The standard metrics are token-based and measure the overlap between your system answer and the reference answer(s). Therefore, we recommend keeping your system generated responses as concise as possible.

Writing your report

We ask you to write a report detailing various aspects about your end-to-end system development (see the grading criteria below).

There will be a **7 page limit** for the report, and we require you to use the [ACL template](#).

Make sure you cite all your sources (open-source models, libraries, papers, blogs etc.) in your report.

Submission & Grading

SUBMISSION CHECKLIST

Submit all deliverables on Canvas:

- Your report.
- A link to your GitHub repository containing your code.^a
- (Optionally) training data you annotated for this assignment.
- Your system outputs on our test set.

^aCreate a private GitHub repo and give access to the TAs in charge of this assignment by the deadline. See the Piazza announcement post for our GitHub usernames.

Your submission should be a zip file with the following structure (assuming the lowercase Andrew ID is ANDREWID).

```
ANDREWID/  
|-- report.pdf  
|-- github.txt  
|-- data/  
|   |-- train (optional)/  
|       |-- questions.txt  
|       |-- reference_answers.json  
|-- system_outputs/  
|   |-- system_output_1.json  
|   |-- system_output_2.json (optional)  
|   |-- system_output_3.json (optional)  
|-- README.md
```

Grading

The following points (max. 100 points) are derived from the results and your report. See the course grading policy.³

- **Submit code (30 points):** submit your code for preprocessing and model development in the form of a GitHub repo. We may not necessarily run your code, but we will look at it. So please ensure that it contains up-to-date code with a README file outlining the steps to run it.
- **Results (30 points):** points based on your system's performance on our private test set. 20 points based on your performance using our metrics,⁴ plus up to 10 points based on level of performance relative to other submissions from the class.
- **Report:** the points below are awarded based on your report.
 - **Data creation (10 points):** clearly describe how you created your data. Please include the following details:
 - * How did you compile your knowledge resource, and how did you decide which documents to include?
 - * How did you extract raw data? What tools did you use?

³Grading policy: <https://cmu-anlp.org/syllabus/>

⁴In general, if your system is generating answers that are relevant to the question, it would be considered non-trivial. This could be achieved with a basic RAG system.

- * What data was annotated for training if any (what kind and how much)?
- * For training data that you did not annotate, did you use any extra data and in what way?
- **Model details (10 points):** clearly describe your model(s). Please include the following details:
 - * What kind of methods (including baselines) did you try? Explain at least two variations (more is welcome). This can include variations of models, which data it was trained on, training strategy, embedding models, retrievers, re-rankers, etc.
 - * What was your justification for trying these methods?
- **Results (10 points):** report raw numbers from your experiments. Please include the following details:
 - * What was the result of each model that you tried on the public leaderboard?
- **Analysis (10 points):** perform quantitative/qualitative analysis and present your findings:
 - * Perform a comparison of the outputs on a more fine-grained level than just holistic accuracy numbers, and report the results. For instance, how did your models perform across various types of questions?
 - * Report your results across at least two variations you tried, including variations of models, which data it was trained on, training strategy, embedding models, retrievers, re-rankers, etc.
 - * Perform an analysis that evaluates the effectiveness of the retrieve-and-augment strategy vs closed-book use of your models.
 - * Evaluate your hybrid retrieval approach by comparing dense-only, sparse-only, and hybrid retrieval performance. Which fusion strategies work best for different types of questions?
 - * Show examples of outputs from at least two of the systems you created. Ideally, these examples could be representative of the quantitative differences that you found above.

Model and Data Policy

To make the assignment accessible to everyone:

- You are only allowed to use models that are also accessible through [HuggingFace](#). This means you may not use closed models like OpenAI models, but you can opt to use a hosting service for an open model (such as the Hugging Face or Together APIs). **The model must have been released before January 1, 2025, and its size must not exceed 32B parameters.**
- You are only allowed to include publicly available data in your knowledge resource and training data.
- You are welcome to use any open-source library to assist your data annotation and model training. For data annotation, you can use tools like Label Studio, Doccano, or similar annotation platforms to create your question-answer pairs efficiently. For model development, you can use standard ML libraries like scikit-learn, PyTorch, or HuggingFace Transformers for any model training, fine-tuning, or evaluation tasks. Make sure you check the license and provide due credit for all tools used.

If you have any questions about whether a model or data is allowed, please ask on Piazza.

FAQ

Q: “There are lots of links and subpages in the webpages, what is exactly the scope for this assignment?”

A: The scope includes the links in this handout and their descendant pages that are specifically relevant to the topics we have listed (e.g. history, events, music, food, sports). In addition, you may also include some PDFs that can be reached from those websites, even if they are not technically descendant pages. Use your best judgment to determine whether a webpage is relevant—a good heuristic is whether we can ask questions about factual content included in those pages.

Q: “Is manual scraping prohibited?”

A: Manual scraping is not prohibited. To what extent you would perform the task manually is up to you.

Q: “What libraries can I use for web scraping and document processing?”

A: You can use standard libraries like Selenium, BeautifulSoup, requests, pdfminer, pypdf, pdfplumber, and similar tools for data collection and preprocessing, as long as you provide proper credit in your report. These are considered basic utilities rather than high-level RAG frameworks.

Q: “What is the date range I should consider for event-based questions?”

A: For any date-based questions specifically about events, we will only ask about events taking place from October 2026 onward; every question with a specific date falls on or after October 22, 2026. Annual and recurring events are also in scope.

Q: “Can I use any closed-source models (OpenAI, Claude, etc.)?”

A: No. You cannot use any closed-source models for any part of the assignment, including embeddings, retrieval, or generation. All models must be open-weight and available through HuggingFace or similar open platforms.

Q: “Can I use LangChain or related frameworks for this assignment?”

A: No, you cannot use LangChain or other RAG frameworks (such as LlamaIndex or similar libraries) that abstract away the core retrieval implementation. The purpose of this assignment is to learn the underlying mechanics of retrieval systems by implementing and integrating them yourself.

Q: “What counts as ‘manual implementation’ for chunking and retrieval?”

A: You need to write the core logic yourself. For chunking: implement the splitting logic, overlap handling, and boundary detection. For hybrid retrieval: implement the integration pipeline, and result processing. You can use libraries as suggested in the sections above (e.g., NLTK for tokenization, NumPy for math operations, FAISS for vector operations, existing BM25 implementations like bm25s or rank-bm25) but not complete retrieval frameworks like LangChain retrievers or high-level RAG pipeline frameworks.

Q: “How do I show that my results are statistically significant?”

A: You can use statistical tests to determine whether performance differences between methods are statistically significant. Examples include paired t-tests (for continuous metrics like F1 scores) and McNemar’s tests (for binary classification accuracy). You can run these tests on multiple metrics, or argue in your writeup that one metric is the best proxy for final system performance. If differences between methods are not statistically significant, interpret what this means for your

system (e.g., a new feature may not actually improve performance despite appearing better).

Q: “If I use late days, will I be evaluated on the same test set as those who do not use late days?”

A: No. We release a different test set for each late day (up to 5), ensuring fairness for everyone in the class. Note that the late penalty itself is the course-wide 5% per unexcused day; see the release policy for late days above.

Acknowledgements

This assignment was based on the Spring 2024 version of this assignment by Graham Neubig and TAs.

References

- Lewis et al., 2021. [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#).
- Touvron et al., 2023. [Llama 2: Open Foundation and Fine-Tuned Chat Models](#).
- Vu et al., 2023. [FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation](#).